

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Javiera Lizana Chartier

12 de septiembre de 2026

02:03

1. Introducción

Este informe estudia empíricamente el rendimiento práctico de algoritmos fundamentales. Por un lado, se aborda el **ordenamiento de arreglos unidimensionales** evaluando MERGESORT, QUICKSORT, PATIENDESORT y STD::SORT. Por otro lado, se examina la **multiplicación de matrices cuadradas**, comparando el enfoque clásico (*naive*) con el método Strassen. El objetivo es medir tiempos de ejecución y consumo de memoria bajo diversos escenarios para contrastar el comportamiento práctico de los algoritmos y su complejidad teórica.

2. Entorno experimental

Los experimentos y mediciones de tiempo de esta Tarea 1 fueron ejecutados bajo las siguientes especificaciones técnicas:

- **Hardware:** Procesador 13th Gen Intel(R) Core(TM) i5-1335U @ 1.30 GHz con 16 GB de memoria RAM.
- **Sistema Operativo:** Windows 11 Pro (versión 25H2) utilizando el subsistema Windows para Linux (WSL) con la distribución Ubuntu 24.04.2 LTS (con aproximadamente 7.6 GiB de RAM asignada) como entorno de ejecución principal.
- **Compilador y Herramientas:** Los algoritmos fueron implementados en C++ y compilados con G++ versión 13.3.0 dentro del entorno Linux. La generación de los casos de prueba y los gráficos de

resultados se realizaron utilizando Python 3.12.3.

Para la evaluación empírica, se utilizaron los datasets sintéticos descritos en el enunciado, generados mediante los scripts proporcionados en la carpeta de la tarea. Para los **algoritmos de ordenamiento** se generaron arreglos de tamaño n para analizar el comportamiento temporal de los algoritmos (Mergesort, PatienceSort, Quicksort y Sort nativo de C++) bajo tres condiciones iniciales: datos generados aleatoriamente, ordenados de forma ascendente y de forma descendente. Por otro lado, para la **multiplicación de matrices**, se utilizaron matrices cuadradas de $n \times n$ pobladas con números enteros aleatorios para comparar el rendimiento temporal y espacial del algoritmo clásico (Naive) frente al algoritmo de Strassen.

3. Resultados y Análisis

Para evaluar el desempeño empírico de las implementaciones, se llevó a cabo una serie de pruebas en las que se midieron dos métricas fundamentales de rendimiento:

1. **Tiempo de Ejecución (ms):** Registrado mediante la librería `std::chrono::high_resolution_clock`, aislando exclusivamente el tiempo de cómputo algorítmico respecto al tamaño de la entrada N .
2. **Uso de Memoria Máxima Residente (KB):** Cuantificado a través de la llamada al sistema `wait4()` y la estructura `rusage (ru_maxrss)`, asegurando registrar el pico de memoria física del proceso hijo independiente para cada ejecución.

La automatización de la toma de muestras y la generación de visualizaciones (gráficos y tablas) se realizó mediante scripts en Python (`plot_generator.py`) integrados en los flujos de compilación (Make), garantizando la trazabilidad y fidelidad entre los datos generados por las implementaciones en C++ y las gráficas presentadas.

3.1. Algoritmos de Ordenamiento

Se evaluaron cuatro algoritmos de ordenamiento con distintas naturalezas teóricas: **MergeSort** (basado en *Dividir y Conquistar*), **QuickSort** (partición *in-place*), **PatienceSort** (basado en la formación de pilas y colas de prioridad), y el algoritmo altamente optimizado de la biblioteca estándar de C++ (`STD::SORT`). Las pruebas consideraron arreglos de tamaños desde $N = 10$ hasta $N = 10^7$ elementos bajo distribuciones ascendentes, descendentes y aleatorias.

3.1.1. Tiempos de Ejecución y Complejidad Temporal

En la Figura 1 y en el Cuadro 1 se observa el crecimiento exponencial logarítmico característico de la familia de ordenamiento óptimo por comparación $\Theta(N \log N)$.

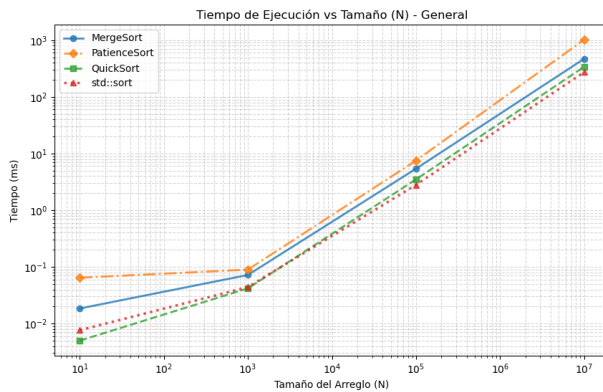


Figura 1: Tiempo de ejecución vs. N para los cuatro algoritmos de ordenamiento.

`std::sort` y `QuickSort` demuestran el mejor rendimiento constante en la práctica. El primero sobresale gracias a sus optimizaciones a nivel de compilador y el uso híbrido de algoritmos, cambiando a *Insertion Sort* para subarrays pequeños y *Heapsort* para evitar peores casos cuadráticos. Por otro lado, *Quicksort* muestra un excelente desempeño promedio, aunque es sensible a la elección del pivote en instancias altamente estructuradas.

`MergeSort` muestra un comportamiento sumamente predecible y estable $\Theta(N \log N)$ en todas las configuraciones de entrada (ascendente, descendente y aleatorio), lo que es fiel al balance teórico de su árbol de recursión y su recurrencia asociada ($T(N) = 2T(N/2) + \Theta(N)$). Sin embargo, experimenta una constante multiplicativa mayor debido a la constante transferencia de datos hacia arreglos auxiliares.

Por último, `PatienceSort` si bien prácticamente opera en $\Theta(N \log N)$, debido a que requiere N inserciones en colas de prioridad basadas en *heaps* de tamaño asintótico menor o igual al número de pilas, en la práctica presenta un tiempo de ejecución superior en entradas masivas ($N = 10^7$), tal como se puede observar en la figura 1, debido a la sobrecarga por la gestión dinámica de múltiples estructuras de datos auxiliares (vectores de punteros y colas de prioridad `std::priority_queue`).

3.1.2. Análisis de Consumo de Memoria

El comportamiento espacial, ilustrado en la figura 2 y resumido en el Cuadro 1, refleja fielmente las diferencias teóricas de espacio auxiliar entre los algoritmos:

Algoritmo	Tipo	Tamaño (N)	Tiempo (ms)	Memoria (KB)
MergeSort	Aleatorio	10^5	5.03	2864
QuickSort	Aleatorio	10^5	2.87	2792
PatienceSort	Aleatorio	10^5	3.18	3196
<code>std::sort</code>	Aleatorio	10^5	2.76	2820
MergeSort	Aleatorio	10^7	528.26	81032
QuickSort	Aleatorio	10^7	286.92	42856
PatienceSort	Aleatorio	10^7	483.75	120332
<code>std::sort</code>	Aleatorio	10^7	214.41	41448

Cuadro 1: Rendimiento de ordenamiento con entradas aleatorias.

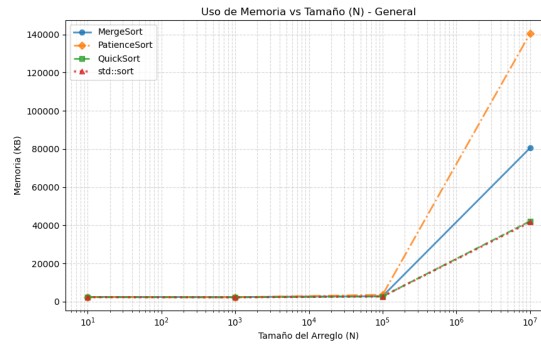


Figura 2: Tiempo de ejecución vs. N para los cuatro algoritmos de ordenamiento.

QuickSort y `std::sort` operan como algoritmos de memoria auxiliar mínima, requiriendo únicamente espacio proporcional a la profundidad del árbol de recursión ($\Theta(\log N)$ en promedio). Para $N = 10^7$, la huella de memoria se mantiene estable en torno a los 41 – 43MB, lo que coincide casi con exactitud con el tamaño base del vector en RAM.

MergeSort exhibe una cota de espacio auxiliar de $\Theta(N)$ al necesitar un arreglo temporal contiguo de igual magnitud para intercalar las mitades,uplicando el consumo de memoria respecto al vector original (alcanzando 81,032KB en $N = 10^7$).

PatienceSort Registra el consumo espacial más elevado de todas las alternativas de ordenamiento (120,332KB en $N = 10^7$). Esto se explica por la estructura enlazada de nodos (*next_node*), los vectores auxiliares de cabeceras de pila y la sobrecarga interna del contenedor de prioridad

3.2. Algoritmos de Multiplicación de Matrices

Se contrastó el algoritmo iterativo clásico (**Naive**) frente al método recursivo de **Strassen** obre matrices cuadradas con dimensiones $N \in \{16, 64, 256, 1024\}$ bajo estructuras densas, diagonales y dispersas

3.2.1. Tiempos de Ejecución y Complejidad Temporal

En la Figura 3 y en el Cuadro 2 se exponen los resultados promedio obtenidos para matrices densas.

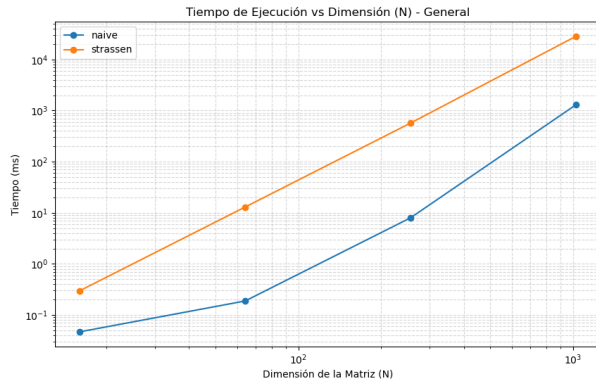


Figura 3: Tiempo de ejecución vs. Dimensión N para multiplicación de matrices (Promedio General).

Dimensión (N)	Tiempo (ms)		Memoria (KB)	
	Naive	Strassen	Naive	Strassen
16×16	0.03	0.29	2 592	2 592
64×64	0.18	11.60	3 040	3 004
256×256	7.79	574.67	3 440	4 512
1024×1024	1 311.43	28 807.50	15 072	46 064

Cuadro 2: Comparativa de rendimiento en matrices densas ($N \times N$).

Los datos empíricos permiten identificar dos fenómenos teóricos fundamentales:

- Dominancia de Naive en dimensiones prácticas ($N \leq 1024$):** El análisis asintótico formal determina que Strassen reduce la recurrencia a $T(N) = 7T(N/2) + \Theta(N^2)$, lo que por Teorema Maestro produce una complejidad subcúbica de $\Theta(N^{\log_2 7}) \approx \Theta(N^{2.8073})$, teóricamente superior a la cota cúbica $\Theta(N^3)$ de Naive. Sin embargo, en la práctica el algoritmo Naive supera con holgura a Strassen en todas las dimensiones evaluadas (por ejemplo, para $N = 256$, Naive toma 7.79 ms contra los 574.67 ms de Strassen)
- Sobrecarga de combinación y asignación recursiva:** Strassen ahorra una multiplicación escalar por nivel a costa de ejecutar 18 sumas y restas de matrices de tamaño $(N/2) \times (N/2)$. En una arquitectura word-RAM real, la subdivisión y combinación de submatrices exige constantes reservas, liberaciones y copias en memoria dinámica. Estas operaciones introducen una constante multiplicativa oculta sumamente pesada que neutraliza la ventaja asintótica en dimensiones intermedias, justificando la necesidad de aplicar umbrales híbridos que deriven el cómputo a Naive cuando N desciende por debajo de un valor límite.

3.2.2. Análisis de Consumo de Memoria

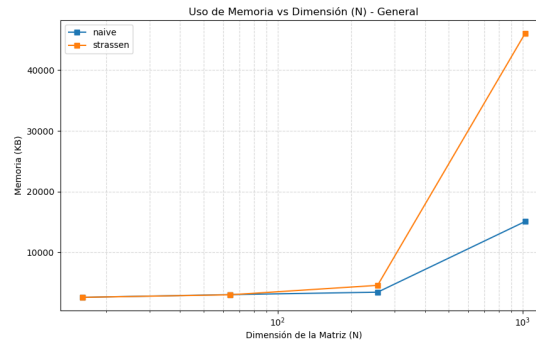


Figura 4: uso de memoria vs. N para las matrices.

De acuerdo con las mediciones ilustradas en la figura 4 sobre el uso de memoria de los algoritmos se tiene que:

- **Naive:** Presenta una utilización espacial óptima $\Theta(N^2)$ circunscrita a la matriz resultante. En $N = 1024$ consume únicamente 15.072 KB de memoria física, manteniendo un espacio de trabajo estático libre de sobrecarga en la pila de llamadas
- **Strassen:** Triplica el uso de memoria residente alcanzando 46.064 KB para $N = 1024$. Este incremento sistemático proviene de la coexistencia de las submatrices temporales $(A, B, \dots, H, P_1, \dots, P_7)$ creadas simultáneamente a lo largo de los distintos niveles de profundidad del árbol de recursión antes de combinarse en los cuadrantes finales

4. Conclusiones

Los resultados demuestran que las cotas de complejidad asintótica proporcionan una orientación indispensable sobre la escalabilidad teórica en el límite ($N \rightarrow \infty$), pero resultan insuficientes por sí solas para predecir el rendimiento absoluto en hardware contemporáneo. Las constantes multiplicativas de bajo nivel, la localidad de acceso a la memoria y el costo de asignación dinámica constituyen factores determinantes en la práctica.

En el ordenamiento unidimensional, la eficiencia *in-place* de QuickSort y la naturaleza adaptativa de `std::sort` las consolidan como las alternativas más convenientes. En multiplicación matricial, la ventaja subcúbica de Strassen permanece eclipsada por sus elevadas constantes ocultas, demostrando que su implementación solo es viable en la práctica cuando se complementa con umbrales de detención híbridos que conmuten a multiplicación iterativa tradicional en tamaños moderados